

VYOM — Complete Presentation & Defense Mastery Guide

Part A: Foundations, Domain Science, Pipeline Deep-Dive, Vocabulary, Features

How to use this document: Read it once fully, start to end, at least 24 hours before you present — not the night before. Then re-read Part B's Q&A section the morning of. The goal isn't memorization; it's that every concept here should feel like something you understand, not something you're reciting. If a sentence doesn't make sense on first read, stop and re-read it until it does — don't skip past it, because that's exactly the sentence a professor will ask about.

1. FIRST: THE MENTAL MODEL YOU NEED

Everything in this project sits inside one field: **image registration**, a sub-field of computer vision and photogrammetry that is decades old and used constantly in medical imaging (aligning MRI scans over time), satellite remote sensing, panorama stitching, and augmented reality. You are not inventing a new field — you are **applying and adapting existing, well-understood techniques to a genuinely hard, specific domain** (the Moon) and **engineering a specific requirement** (uniform match distribution) that generic tools skip. Say this to yourself until it's automatic: **"We are not claiming to have invented image matching. We are claiming to have engineered it correctly for the Moon's specific challenges, and validated it honestly."** That single sentence, said with confidence, defuses most hostile framing before it starts.

2. FOUNDATIONAL CONCEPTS — THE CRASH COURSE

2.1 What "image registration" formally means

Given two images of the same scene, find a **geometric transformation** T such that applying T to the source image makes its content spatially coincide with the reference image. There are two broad families of methods:

- **Area-based / intensity-based registration:** compares raw pixel intensities directly (e.g., cross-correlation, mutual information). Works well when both images have similar brightness/contrast, but breaks down badly when illumination differs a lot — which is exactly your Moon problem.
- **Feature-based registration:** first extracts a sparse set of distinctive points (keypoints) from each image, describes each one with a numeric "fingerprint" (descriptor), matches fingerprints between images, then fits a transform to the matched point pairs. **This is the family your entire project belongs to**, and it's the correct choice specifically *because* it's more robust to illumination and viewpoint differences than area-based methods.

2.2 Transformation models (know these cold — "why homography and not affine" is a very likely question)

Model	Degrees of freedom	What it can represent	Minimum point correspondences needed
Translation	2	Pure shift	1
Rigid/Euclidean	3	Shift + rotation	2
Similarity	4	Shift + rotation + uniform scale	2
Affine	6	Shift + rotation + scale + shear (parallel lines stay parallel)	3
Homography / Projective	8	Full perspective distortion (parallel lines can converge)	4

Why homography is the right default for this project: two cameras looking at the *same planar (or near-planar) surface* from *different positions* are related by a homography — this is a fundamental result in projective geometry (see Hartley & Zisserman, *Multiple View Geometry in Computer Vision*, the standard reference for this). Since a small lunar image crop is locally close to flat (relative to orbital altitude), homography is a reasonable model. **The honest caveat you must know:** this assumption weakens over larger areas with real elevation relief (crater walls, mountains) — true accuracy there requires **orthorectification** using a **Digital Elevation Model (DEM)**, which is a deliberate simplification you are *not* implementing, and you should say so proactively if asked "does this work perfectly everywhere on the Moon."

How homography is actually computed: each matched point pair gives 2 linear equations; with the unknowns arranged into a 3×3 matrix (9 entries, 8 free because it's defined up to

scale), 4 point correspondences give exactly enough equations to solve it via the **Direct Linear Transform (DLT)** algorithm, solved with Singular Value Decomposition (SVD). This is precisely why RANSAC (next section) uses a **minimal sample of 4 points per iteration** for homography.

2.3 RANSAC — explain this perfectly, it WILL be asked

RANSAC (RANdom SAMple Consensus), introduced by Fischler & Bolles (1981), is an algorithm for fitting a model in the presence of many bad data points (outliers), without knowing in advance which points are bad.

How it actually works, step by step:

1. Randomly pick the minimal number of point-matches needed to define the model (4 for homography).
2. Compute the candidate transform from just those points.
3. Apply that transform to *every* matched point, and measure how far off each predicted point lands from its actual match (the **reprojection error**).
4. Any match within a chosen distance threshold (e.g., 3 pixels) is called an **inlier** for this candidate; everything else is an **outlier**.
5. Repeat steps 1–4 many times (each repetition uses a different random minimal sample).
6. Keep the candidate transform that produced the **largest inlier set** (the "consensus set").
7. Optionally, refit the final transform using *all* inliers together via least-squares, for a more accurate final answer.

Important nuance to know: because step 1 involves *random* sampling, running RANSAC twice on the same data can give very slightly different results — this is expected and normal, not a bug. If a judge asks "why did your numbers change slightly just now," this is the correct, confident answer.

2.4 Feature detectors — know the real difference, not just the names

- **SIFT (Scale-Invariant Feature Transform)**, Lowe (2004): builds a "scale space" by blurring the image at increasing levels and taking differences between blur levels (**Difference-of-Gaussians**, an efficient approximation of a more expensive operator). Keypoints are local extrema in this scale space — points that are stable across zoom levels. Each keypoint gets an orientation (for rotation invariance) and a 128-number descriptor built from local gradient histograms. **Strength:** robust to scale and rotation. **Weakness:** built from *intensity gradients*, so it degrades when brightness/contrast patterns change nonlinearly — exactly what happens under different sun angles.
- **AKAZE:** builds its scale space using **nonlinear diffusion** instead of Gaussian blurring — this preserves edges better than SIFT's blurring while smoothing noise, and is computationally

faster. Still fundamentally gradient/intensity-based, so it shares SIFT's illumination weakness, just with better speed and slightly different edge behavior.

- **Your phase-congruency-based matcher:** this is the conceptually different one — see 2.5.

2.5 Phase congruency — your headline scientific claim, explain it precisely

Ordinary edge/corner detectors (including SIFT/AKAZE) look at **gradient magnitude** — literally “how fast does brightness change here.” The problem: brightness itself is exactly what changes between a lunar-noon photo and a low-sun-angle photo, so gradient magnitude is unreliable across illumination changes.

Phase congruency, developed by Peter Kovesi, is built on a different physical idea, called the **local energy model**: it decomposes the image into different Fourier frequency components (using a bank of **log-Gabor filters** at multiple scales and orientations) and looks for locations where a large number of these frequency components are **all in phase with each other** — i.e., their peaks and troughs line up at the same spot. Points of high phase congruency correspond to genuine structural features (edges, lines, corners) **because a real edge produces phase alignment across many frequencies regardless of the image's overall brightness or contrast**. This is the precise, correct reason it's illumination-invariant: **it responds to the shape of the signal, not its amplitude**. This is exactly the mathematical basis of the RIFT (Li, Hu & Ai, 2020, IEEE Transactions on Image Processing) and RIFT2 methods your matcher is inspired by — it was originally developed for matching optical images against SAR (radar) and infrared images, which have completely different radiometric behavior from each other, which is precisely analogous to matching Chandrayaan-2 optical images against differently-illuminated reference images.

Be ready for this follow-up: “does phase congruency have any downside?” Yes — it's more computationally expensive (multiple filters at multiple scales/orientations computed via FFT, versus a simple gradient), which is exactly why you had to optimize it (downsampling before computing it, using the cheaper monogenic-signal variant, padding to FFT-efficient sizes). This is a good, honest, technically fluent answer.

2.6 CLAHE and histogram matching — your simpler preprocessing tools

- **CLAHE (Contrast Limited Adaptive Histogram Equalization):** divides the image into small tiles, equalizes the brightness histogram *within each tile* independently (so it adapts to local contrast rather than one global adjustment), and **clips** the histogram before equalizing to avoid amplifying noise in flat, low-contrast regions (like deep shadow). This helps reveal detail in both very bright and very dark parts of the same image.
- **Histogram matching:** reshapes one image's brightness distribution (its histogram) to match another image's distribution, by matching their cumulative distribution functions. Useful when

one sensor is systematically brighter/darker than another (e.g., IIRS vs. WAC), independent of local shadow structure.

2.7 Metrics — say these with precision

- **RMSE (Root Mean Square Error), in pixels:** for every inlier match, compute how far the transformed source point lands from its true matched reference point (the reprojection error), square each of those distances, average them, then take the square root. Lower = better geometric accuracy. **"Sub-pixel accuracy" specifically means this value is below 1.0 pixel.**
- **Inlier ratio:** inlier matches ÷ total candidate matches found before filtering — measures how "clean" the raw matching was.
- **Inlier count:** the raw number of surviving good matches — matters because too few matches (even if all correct) makes the fitted transform statistically fragile.
- **Distribution/coverage score (your own addition):** fraction of a grid overlaid on the image that contains at least one inlier match — this is *not* a standard textbook metric; it's something you added specifically because the problem statement explicitly asked for uniform distribution, and standard tools/papers don't typically report it. **This is genuinely your own contribution — say so explicitly, it's one of your strongest points.**

3. WHY THE MOON IS A GENUINELY HARDER PROBLEM THAN EARTH REMOTE SENSING (domain science — this is where you'll sound like you actually know the field)

Most registration research (and most existing tools) was built for Earth imagery, medical imaging, or aerial photography. The Moon breaks several assumptions those tools rely on:

- **No atmosphere → no diffuse skylight.** On Earth, shadows are never fully black because scattered skylight fills them in. On the Moon, there is no atmosphere to scatter light, so shadows are close to pure black with essentially zero recoverable detail — a much harsher, more binary illumination problem than anything in terrestrial remote sensing.
- **Lunar regolith is not a simple diffuse (Lambertian) reflector.** Its brightness depends strongly on the **phase angle** — the angle between the incoming sunlight, the surface, and the observing camera. This is described in planetary science by photometric functions such as the **Lommel-Seeliger law** or the more complete **Hapke photometric model**, and includes a well-documented **opposition effect (opposition surge)** — a sharp brightness spike when the phase angle approaches zero (sun directly behind the observer). This means the *same* patch of ground can look dramatically different in brightness not just from shadows, but from the fundamental physics of how regolith scatters light — a phenomenon simple

contrast/histogram correction (CLAHE) does not model at all. **Mentioning this distinction — that your preprocessing is a practical approximation, and a true production system would apply photometric correction using a model like Hapke's — is one of the most scientifically credible things you can say, because it shows you understand the limits of your own approach.**

- **Repetitive, self-similar terrain.** Craters look like other craters. This causes a real, well-known failure mode in feature matching called **feature ambiguity or aliasing** — the algorithm confidently matches point A to the *wrong* crater rim that happens to look similar, rather than the correct one, especially in featureless mare regions. This is why RANSAC's *geometric* consistency check (not just descriptor similarity) is essential — it can reject a "confident-looking" but geometrically inconsistent match.
- **Orders-of-magnitude scale disparity.** A 0.25 m/pixel OHRC image versus an 80 m/pixel IIRS image differ by roughly a factor of 320 in linear resolution — far beyond the scale ranges most general-purpose registration benchmarks are tested on (typically 2-5x).
- **True multimodality.** IIRS is hyperspectral (many narrow wavelength bands beyond visible light); comparing it to a simple panchromatic (single broadband) reference image is a genuinely different problem from comparing two panchromatic images — you are implicitly collapsing IIRS's spectral cube into a representative 2D image (e.g., a single band or a combined representation) before matching, and you should be explicit and deliberate about which approach you used and why, since this is a place a domain-savvy judge could probe.

4. YOUR PIPELINE, STAGE BY STAGE, WITH THE "WHY" FOR EACH STAGE

1. **Load & resample** — brings both images to comparable resolution before anything else, because feature detectors have limited scale invariance range; without this step, matching across a 320x resolution gap would fail almost entirely regardless of algorithm.
2. **Preprocessing (CLAHE / histogram matching / phase congruency map)** — reduces radiometric (brightness/contrast) differences *before* feature detection, since every downstream step depends on features being detectable in the first place.
3. **Feature detection & description (SIFT / AKAZE / phase-congruency-based)** — extracts a sparse, distinctive, matchable representation from each image; algorithm choice here is your main "illumination robustness" lever.
4. **Descriptor matching + ratio test** — pairs up candidate corresponding points based on descriptor similarity; the ratio test (comparing the best match's distance to the second-best) rejects ambiguous matches where two candidates look almost equally similar — a classic technique from Lowe's original SIFT paper.

5. **RANSAC + homography fitting** — separates geometrically consistent matches (inliers) from coincidental/incorrect ones (outliers), and produces the transform that aligns the two images.
6. **Grid-based uniform distribution enforcement** — your specific engineering addition: prevents the final match set from being dominated by one easy, high-contrast region, which would make the fitted transform less reliable across the *rest* of the image (a homography fit from clustered points can be locally accurate but globally poor — this is a real, well-known issue in photogrammetry called **poor geometric conditioning**).
7. **Warping** — applies the estimated transform to resample the source image onto the reference image's pixel grid, using interpolation (bilinear) to compute pixel values at non-integer coordinates.
8. **Metrics computation** — quantifies what happened, honestly, rather than presenting only a visual "it looks aligned."

5. SCIENTIFIC VOCABULARY GLOSSARY — USE THESE CORRECTLY, NOT AS BUZZWORDS

(Each term includes a precise definition. Do not use a term unless you could explain it if asked to elaborate — a misused technical term is worse than a plain-English sentence.)

Term	Precise meaning
Ground Sampling Distance (GSD)	The real-world distance on the ground represented by one pixel (e.g., OHRC's GSD is 0.25 m/pixel).
Radiometric variation	Differences in pixel brightness/contrast between images (illumination, sensor sensitivity) — as opposed to <i>geometric</i> variation (position/shape).
Geometric variation	Differences in viewpoint, scale, or orientation between images.
Multimodal registration	Registering images from fundamentally different sensor types (e.g., optical vs. hyperspectral vs. radar), which have different radiometric behavior even of the same scene.
Scale-space representation	Representing an image at multiple levels of blur/detail simultaneously, to detect features that are stable across zoom levels.
Difference-of-Gaussians (DoG)	An efficient approximation used by SIFT to find scale-space extrema (candidate keypoints).
Phase congruency	A measure of how strongly different frequency components of a signal align in phase at a point — high phase congruency indicates a genuine structural feature, independent of brightness/contrast.
Log-Gabor filter	A frequency-domain filter (symmetric on a log-frequency scale) used to decompose an image into different frequency/orientation components, the building block of phase congruency computation.
Monogenic signal	A more efficient, single-filter-per-scale alternative to the full multi-orientation phase congruency computation, based on Riesz transforms.
Homography / projective transform	An 8-degree-of-freedom transformation modeling how two views of the same planar surface relate.
Direct Linear Transform (DLT)	The standard linear-algebra method (via SVD) for solving for a homography from point correspondences.

Term	Precise meaning
RANSAC / consensus set / inlier threshold	See section 2.3 — the outlier-robust model-fitting algorithm and its core terms.
Reprojection error	The pixel distance between a transformed point and its actual matched location — the basis of RMSE.
Sub-pixel accuracy	Registration accuracy finer than the width of one pixel (RMSE < 1.0).
Orthorectification	Correcting an image so every pixel represents its true ground position by removing distortion from terrain relief and sensor viewing geometry, typically using a Digital Elevation Model (DEM).
Digital Elevation Model (DEM)	A raster representing ground elevation, used for orthorectification and 3D terrain correction.
SPICE kernels	NASA/JPL's standard system of files describing a spacecraft's precise position, orientation, and instrument pointing over time — required for rigorous photogrammetric georeferencing.
Photometric function (Hapke model / Lommel-Seeliger law)	Mathematical models describing how a planetary surface's brightness depends on illumination and viewing geometry (phase angle) — relevant because lunar regolith is not a simple diffuse reflector.
Phase angle (photometry)	The angle at the target between the direction to the light source (Sun) and the direction to the observer — governs regolith brightness behavior, including the opposition effect.
Opposition effect / opposition surge	A sharp brightness increase observed on regolith surfaces (like the Moon) when the phase angle approaches zero.
Bundle adjustment	The production-grade technique (used in real photogrammetric pipelines, not implemented here) that jointly refines camera poses and 3D point positions across many images to minimize total reprojection error.

Term	Precise meaning
Non-maximum suppression / Adaptive Non-Maximal Suppression (ANMS)	Techniques for thinning out densely clustered keypoints to keep only the strongest/most spatially spread ones — conceptually related to your grid-based distribution enforcement.

6. WHAT YOU'VE ACTUALLY BUILT (your "wow" features) vs. WHAT'S NEXT

(Confirm each of these against your actual final build before presenting — mark anything that didn't make it into the final version and remove it from your talking points; never claim a feature that isn't real.)

Genuinely strong, defensible features (lead with these)

- **A working full-stack application** — React/Vite frontend, FastAPI backend, real Python/OpenCV registration engine underneath — not a mockup, not slides pretending to be software.
- **Multiple selectable, real matching algorithms** (SIFT, AKAZE, your phase-congruency-based mode) running against the *same* pipeline, with a live side-by-side comparison view — this single feature does more to prove "we understand trade-offs" than anything else in the app.
- **A specific, engineered answer to the problem statement's literal wording** — grid-based uniform match distribution — that most public tools and even some research implementations don't include.
- **A synthetic ground-truth validation method** — you can produce an *objectively verifiable* accuracy number, not just a plausible-looking image overlay, because you know the exact transform you applied.
- **Real quantitative metrics displayed live** (RMSE, inlier ratio, inlier count, distribution score) rather than a qualitative "looks good."
- **Honest, visible failure handling** — if a pair doesn't match well, the app says so rather than faking success.

Planned / future scope (be ready to discuss, but clearly label as not-yet-built)

- **A trained, learned component** — most credibly, a lightweight **crater detector** trained on real Chandrayaan-2/LRO imagery using existing public crater catalogs as automatic labels (no

manual labeling needed), used as an illumination-invariant landmark for matching, layered on top of your current geometric pipeline rather than replacing it.

- **Photometric correction using a proper reflectance model (Hapke/Lommel-Seeliger)** instead of generic contrast equalization, for scientifically rigorous illumination normalization.
- **SPICE-based georeferencing and DEM-based orthorectification**, to move beyond the flat-terrain homography assumption toward true photogrammetric accuracy.
- **An optional deep-learning matcher (SuperGlue-class)** for cases where classical/phase-congruency methods fail (e.g., extreme polar illumination), clearly labeled as using a general-purpose pretrained model, not lunar-specific.
- **Batch processing and a persistent run-history dashboard**, for handling many image pairs and comparing results over time.
- **Radar (DFSAR) support**, extending beyond optical/hyperspectral sensors.

(Continued in Part B: how to explain this to judges, the full risk map, the definitive answer to the "pretrained model" objection, and the complete Q&A bank.)

VYOM — Complete Presentation & Defense Mastery Guide

Part B: Explanation Scripts, Risk Map, The Pretrained-Model Objection, Master Q&A, Final Checklist

7. HOW TO EXPLAIN THE PROBLEM AND SOLUTION — LAYERED SCRIPT

Use this two-layer structure for every explanation: **plain analogy first, then precise technical version**. Never skip straight to jargon, and never stay only at the analogy level if pressed further.

The problem, layer 1 (analogy — use this to open)

"Imagine two people photograph the same mountain — one with a drone at noon, sharp shadows, extreme zoom; another from a hillside at sunset, long shadows, wider view. If you wanted software to automatically find the same rock in both photos, it has to see past the lighting and zoom differences. That's what we're doing, except the 'mountain' is the Moon,

photographed by an Indian spacecraft and by a NASA spacecraft, months or years apart, at completely different resolutions."

The problem, layer 2 (precise)

"Formally, this is a **multimodal image registration** problem: given a source image (Chandrayaan-2's OHRC, TMC-2, or IIRS) and a reference image (LRO's NAC/WAC or SELENE's imagery) of overlapping lunar terrain, find corresponding points and the geometric transform that aligns them — despite **radiometric variation** from differing sun angle (illumination), **geometric variation** from differing viewpoint, and **scale variation** spanning over two orders of magnitude in ground resolution."

The solution, layer 1 (analogy)

"Instead of comparing how bright each spot is — which changes with lighting — we also compare the *shape* of features, using a method that stays consistent even when brightness changes completely. Then we use a voting system to throw out any matches that don't geometrically agree with the rest, and we make sure the matches we keep are spread evenly across the photo, not clumped in one easy corner."

The solution, layer 2 (precise)

"Our pipeline resamples both images to a common scale, applies illumination-normalizing preprocessing, then detects and matches features using either classical gradient-based descriptors (SIFT/AKAZE) or a **phase-congruency-based** descriptor for illumination-invariant matching, inspired by the published RIFT/RIFT2 approach. We filter matches using **RANSAC** to estimate a homography, with an added **grid-based redistribution step** that explicitly enforces spatial coverage — directly answering the problem statement's requirement for uniform match distribution. The source image is then warped onto the reference frame, and we report **RMSE, inlier count, inlier ratio, and a distribution score** as objective evaluation metrics."

8. THE CRITICAL RISK MAP — WHERE YOU CAN GET CAUGHT OUT, AND THE CORRECT ANSWER FOR EACH

Risk area	What could go wrong if asked	The correct, honest answer to give
Overclaiming "generic solution"	Judge asks "does this work on ANY	"It's generic in that it isn't hardcoded to one image pair — it works across the sensor pairs we tested

Risk area	What could go wrong if asked	The correct, honest answer to give
	lunar image pair?"	(name them). We haven't validated it across the entire lunar surface, especially permanently-shadowed polar regions, which we've explicitly scoped as future work."
Confusing "algorithm" with "trained model"	Judge asks "so did you train anything or not?"	"SIFT, AKAZE, RANSAC, and our phase-congruency matcher are fixed mathematical algorithms — no training data involved. The only pretrained component is our optional SuperGlue mode, which we treat as a secondary comparison, not our core method."
Overclaiming accuracy	Asked "how accurate is it, really?"	Only ever state the actual measured number from your own test runs. If you don't have one memorized cold for both the synthetic and real pair, look it up before presenting — do not estimate live.
The synthetic validation being mistaken for real-world proof	Judge asks "so is this number from a real Chandrayaan-2 image?"	"Our lowest, most confident RMSE is from a synthetic pair where we know the exact ground-truth transform, which lets us validate the algorithm's correctness objectively. On real cross-mission data, our number is [X] — higher, and more representative of real-world performance." Never let these two numbers get conflated.
Homography's flat-terrain assumption	Judge asks "does this work near a crater wall/mountain?"	"Homography assumes locally near-planar terrain, which holds reasonably well for small crops. For larger areas with real elevation relief, true accuracy needs orthorectification using a Digital Elevation Model — that's a known limitation we didn't implement."
RANSAC's randomness	"Why did the number change slightly when you ran it again?"	"RANSAC uses random sampling internally, so very small variation run-to-run is expected and normal — it's not an error."

Risk area	What could go wrong if asked	The correct, honest answer to give
IIRS being hyperspectral	"How do you match a hyperspectral cube to a single-band image?"	Know exactly which band(s)/representation you actually used in your implementation and say so wasn't handled carefully in the build, say plainly: "For the MVP we used [specific band/approach] as a practical simplification; a fuller approach would consider spectral matching techniques across bands."
Compute cost of the phase-congruency mode	"Why is this mode slower?"	Explain the log-Gabor multi-scale/multi-orientation filtering honestly (section 2.5), and mention the specific optimizations you applied (downsampling before computing it, monogenic-signal variant, FFT-efficient padding).
The "why not just use existing software" challenge	"ISRO/USGS already has tools for this (ISIS, etc.) — why does this matter?"	"Full photogrammetric suites like ISIS3 require SPICE metadata, orbital data, and specialist expertise to operate — they solve the <i>general</i> problem at production scale. We built something scoped specifically to the <i>matching/registration</i> step, deployable quickly, and specifically engineered around the illumination and distribution requirements named in this exact problem statement."
Team knowledge gaps	Only one teammate can explain a given part	Do the "explain a random part cold" rehearsal round with your whole team before presenting (see the checklist below) — this is the single most common way teams get caught.

9. THE DEFINITIVE ANSWER TO "YOU JUST USED PRETRAINED MODELS/LIBRARIES"

This objection will very likely come up again in some form. Here is the complete, layered answer:

Opening correction (do this calmly, not defensively): "Respectfully, only one optional mode in our system uses a pretrained model — everything else, including our main illumination-invariant method, is algorithm engineering with zero training data involved. SIFT and AKAZE are fixed mathematical formulas, not trained models. Our phase-congruency-based matcher is also a fixed signal-processing algorithm, not a neural network."

The "why doesn't ISRO already have this" point: "The most recent published research from ISRO's own Space Applications Centre on this exact problem — comparing SIFT, ASIFT, AKAZE, RIFT2, and SuperGlue on Chandrayaan-2 data — also does not train a new model from scratch. That's because there isn't yet a large, labeled dataset of verified cross-mission lunar tie-points to train on. We built on the same category of proven methods that current research uses, and specifically engineered a requirement — uniform match distribution — that the published comparison doesn't address."

The "what would 'training our own model' actually look like, and why we didn't force it" point: "Training a meaningful custom model needs labeled data. There's no ready-made 'here are 10,000 correct lunar tie-points' dataset. The realistic path to a real trained component — which we've scoped as our next step, not done yet — is training a lightweight crater detector using existing public crater catalogs as automatic labels, since craters are the one feature whose *shape* survives illumination changes. That's a real, achievable training task, unlike trying to train a generic matcher from nothing."

Close with ownership, not defensiveness: "We used AI coding tools to build faster, the way most engineers do now — but every decision about *which* algorithm to use, *why*, and how to validate it honestly was ours, and we can walk through any part of it in detail."

10. MASTER Q&A BANK

A. Foundational computer vision / image processing (professors love these — they test real understanding, not project trivia)

Q: What is image registration, in one sentence? A: Finding the geometric transformation that aligns two images of the same scene into a common coordinate system.

Q: What's the difference between feature-based and intensity-based (area-based) registration? A: Feature-based extracts and matches distinctive points first, then fits a transform; intensity-based directly compares raw pixel values (e.g., cross-correlation) without extracting features. Intensity-based is simpler but fails badly under large illumination differences — which is why we chose feature-based.

Q: Explain RANSAC in your own words. A: (Use section 2.3 — minimal sample, fit, count inliers by reprojection error, repeat, keep the best consensus set, optionally refit with all inliers.)

Q: What is a homography, and how many points do you need to compute one? A: An 8-degree-of-freedom transform modeling how two views of a planar surface relate; you need a minimum of 4 point correspondences to solve for it.

Q: Why is SIFT called "scale-invariant"? A: Because it detects keypoints as stable extrema across a multi-level blurred scale-space representation of the image, so the same physical feature is detected consistently whether the image is zoomed in or out.

Q: What does "sub-pixel accuracy" actually mean? A: That the RMSE of the registration is less than one pixel — i.e., on average, matched points land within less than a single pixel's width of their true corresponding location after alignment.

Q: What's the difference between accuracy and precision in this context, if asked? A: Accuracy here (RMSE) measures how close matched points are to their true correspondence after the transform; it doesn't by itself measure whether matches were found consistently across the whole image (that's what our distribution score separately addresses).

B. Domain-specific (Moon/remote sensing)

Q: Why is registering lunar images harder than registering Earth satellite images? A: (Use section 3 — no atmosphere/no skylight-filled shadows, non-Lambertian regolith reflectance and phase-angle dependence, repetitive crater terrain causing feature ambiguity, and larger scale disparities between missions.)

Q: What is the "opposition effect" and is it relevant here? A: A sharp brightness increase in regolith surfaces observed near zero phase angle; it's relevant because it shows lunar surface brightness depends on more than simple shadows — a full photometric correction (e.g., Hapke's model) would need to account for it, which we've noted as a limitation/future improvement.

Q: How do OHRC, TMC-2, and IIRS differ? A: (Recite exact specs from your dataset table — resolution, swath, spectral range for each — know these numbers cold.)

Q: Why LRO NAC as the main reference, and not something else? A: It offers comparable or finer resolution to Chandrayaan-2's OHRC, is publicly and freely accessible via NASA's PDS archive without the access friction of some other sources, and is the reference used in the most relevant published benchmark study we grounded our approach in.

C. Project-specific

Q: Walk me through your pipeline end to end. A: (Use section 4 — resample, preprocess,

detect/match, RANSAC + homography, grid-based distribution enforcement, warp, metrics.)

Q: What's actually novel about what you built, versus existing tools/research? A: The explicit engineering of uniform match distribution (a literal requirement in the problem statement that generic tools and even the closest published benchmark study don't address), and a single tool benchmarking multiple lunar-tuned matchers side by side with objective, ground-truth-validated metrics.

Q: How did you validate your accuracy claims? A: We built a synthetic pair by applying a *known* transform to a real lunar image, so we could compute an objective RMSE against ground truth, in addition to testing on real cross-mission data.

Q: What happens if the two images don't actually overlap? A: The pipeline detects an unreliable/near-zero inlier result and the application reports this honestly rather than displaying a fabricated alignment.

Q: Why did you choose a React + FastAPI architecture? A: To keep the registration engine (in Python, where the computer-vision ecosystem lives) fully decoupled from the interface, so the algorithm logic is independently testable and the UI can evolve separately — we also moved to this from an earlier Streamlit prototype once we needed more control over the interface and to solve deployment weight issues with heavier libraries.

Q: What are your current limitations? A: (Use section 6's future-scope list, stated plainly — no DEM-based orthorectification, no photometric reflectance correction, limited to a curated set of tested pairs, no radar/DFSAR support, and only one optional pretrained component.)

D. The tough/adversarial questions

Q: "This is just SIFT with extra steps — what's actually yours?" A: The specific combination and engineering choices — the phase-congruency-based illumination-invariant matcher implementation, the grid-based uniform distribution enforcement, the synthetic ground-truth validation methodology, and the comparative benchmarking tool — is our integration and engineering work, applied to a specific, real, hard problem. We're not claiming to have invented feature matching; we're claiming to have engineered a correct, validated solution to this specific problem statement.

Q: "Why should we believe your numbers aren't cherry-picked?" A: Show the Compare Algorithms view live, on more than one pair, including a harder case where results are visibly less perfect — showing an imperfect real result live is more convincing than only showing your best case.

Q: "Could a first-year student build this by just calling OpenCV functions?" A: The core functions (SIFT, RANSAC) are indeed available in OpenCV — that's true of almost all applied computer vision work, including in industry and research. The engineering is in *how* they're

combined, tuned, and validated for this specific, hard domain, and in the parts we implemented ourselves outside of any library (the phase-congruency matcher's application here, the distribution enforcement, the validation methodology).

Q: "What would you do differently if you had one more month?" A: Train the crater-detector landmark model described in section 6, and implement a proper photometric correction step using a reflectance model like Hapke's instead of generic contrast equalization.

E. Future scope / scalability

Q: How would this scale to production use at ISRO? A: (Use section 6/blueprint's scalability path — SPICE-based georeferencing, DEM-based orthorectification, batch processing, integration with ISRO's own data pipelines.)

Q: Would a bigger model just solve this better? A: Not automatically — the bigger limitation right now is the lack of a large labeled lunar tie-point dataset to train on, not a lack of model capacity. A trained model is only as good as its training data, and building that dataset (e.g., via crater catalogs) is itself real, necessary work — which is exactly our proposed next step.

11. FINAL PRE-PRESENTATION CHECKLIST

- ☐ Every team member has done the "explain a random pipeline stage cold" drill, out loud, to the rest of the team
- ☐ You can state your actual measured RMSE (synthetic AND real) from memory, without checking notes
- ☐ You've re-read section 9 (the pretrained-model objection) enough times that your answer sounds calm, not rehearsed-defensive
- ☐ You know the exact OHRC/TMC-2/IIRS/NAC specs (resolution, swath) cold
- ☐ You've picked which two demo pairs you'll show live (one clean case, one harder case where classical methods visibly struggle and your phase-congruency mode does better)
- ☐ Backend and frontend are both pre-started and warmed up before you go up
- ☐ You have a backup screen recording of a successful run
- ☐ You've timed a full run-through against your actual 8-10 minute limit, out loud, with a clock
- ☐ Every teammate knows they may get a direct, individual question — and is ready to answer, not defer to one "main presenter"
- ☐ You've decided, as a team, who answers the "pretrained model" question if it comes up again, and that person has read section 9 twice